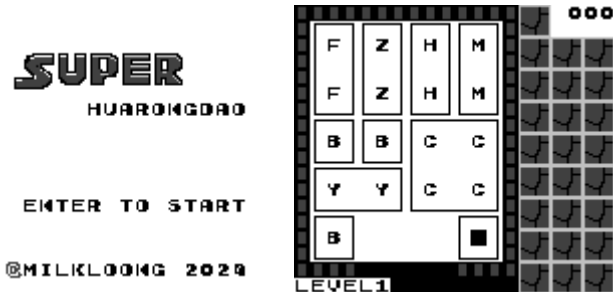


Assembly Game - Hua Rong Dao

by team Milk Dragon

Introduction

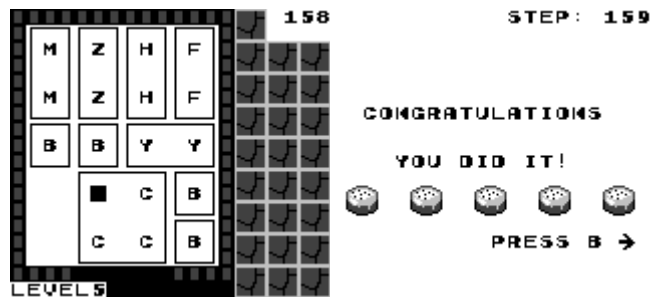


Huarongdao is a puzzle game. The winning condition is to help Cao Cao (block with alphabet c) find a way out (reach the black part at the bottom of the puzzle map). There're several blocks that Cao Cao cannot cross over, but you can move the blocks to make free spaces. Think of how to use these blocks wisely to make a way!

The game has a counter to memorize total steps to move block. Each move of a block counts 1 step. You can see how many step you have used from the counter. Watch out! If you use more than 255 steps, Cao Cao have no more time to get out and stuck in Huarongdao forever...

Since the puzzle has a solution, there should be a best solution with least steps. Try to solve the puzzle first, if you have already solved it, optimize it to make the solution better.

There would be 5 maps in our game, from easy to hard. Get used to the game by starting the first puzzle and try the best to challenge the rest!



Don't worry, you can see all puzzles even if you haven't solved any one of them, just press B to see next puzzle.

Main Idea

Display title screen at first. Check if start button is pressed to begin game.

Initialize `fsmState` as 0 if not selected, 1 if do.

Run loops to read keys according to `fsmState`.

At any case, if select button is pressed, reinitialize everything to restart the current map; If button B is pressed, increase map number or set it to 1 if it reaches the last, and reinitialize object with given map number.

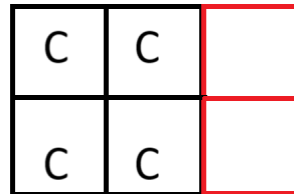
0: Check if button A is pressed, if do, set `fsmState`. If not, check if pressed key is a direction or some other input. For direction input, move cursor with given direction.

1: Move object with given direction. Reset `fsmState` after next direction button is pressed.

For moving, consider moving right, since other 3 directions have similar structure with moving left.

We have to consider 7 different cases, Bing (small square), Cao Cao (big square), Zhang Fei, Huang Zhong, Ma Chao, Zhao Yun (4 vertical rectangles), Guan Yu (horizontal rectangle).

Now we consider the case Cao Cao. If Cao Cao wants to move right, then these spaces for Cao Cao should be free space:



To check if a position is free space, use a function `GetTileByPixel` to get tile id assigned to the position. Note that the destination is seen as free space for Cao Cao, but not for others.

The functionality and progress of `GetTileByPixel` will be explained in the "Function" part, beyond "Main Idea" part.

Run `GetTileByPixel` for northeast and southeast corners. If tile id corresponding them have match some block object, then there're some positions that it is not free space, and skip.

If there're all free spaces, use `.updatecaocaobackground` to update background corresponding to Cao Cao.

The functionality and progress of `.updatexxxbackground`s will be explained in the "Function" part, beyond "Main Idea" part.

As Cao Cao get out only by moving down to the screen, when Cao Cao moves down, also check if tiles at destination is now Cao Cao tile. If so, load winning page to screen.

Increase step counter after every available move. Counter relative functions will be explained in "Function" part.

Data

Cursor

Cursor information is stored as the first object in OAM.

Background and Blocks

Cursor and center positions of movable blocks are stored in the array in OAM.

Background and whole pictures of movable blocks are all stored in VRAM.

We have 7 background map in `data.inc`: 5 maps, title screen and winning screen.

32 is the width of background map to store tiles. We store background information from up to down, then left to right. So $32Y+X$ points to the background in (X,Y).

Current map number is stored in variable "gotostate".

Counter

Counter number itself is stored in global variable "counter" and its digits are stored separately in "buffer".

Numbers that displayed on the screen, which are stored in VRAM, are read from "buffer".

Function

GetTileByPixel

Functionality: Convert a pixel position to a tile map address.

Progress:

Assume we have x, y stored in OAMRAM, then $b = x - 8$, $c = y - 16$ because the origin, namely the northwest corner of the screen, is (16, 8) in OAMRAM.

A tile is 8 times 8, so b, c are multiples of 8 (that is, $b=8X$, $c=8Y$). To obtain the mathematical position (X,Y), divide (b,c) by 8.

For c, we wish to perform right shift operation for it to obtain Y. But we have to multiply Y by 32 afterwards, then there're some operations that can be optimized.

Instead to get Y by `sr1` 3 times and multiply Y by 32, we remain 8Y and multiply 8Y by 4. 8Y is obtained by `and a, %11111000`, notice that `%00001000` = 8 so valid part of Y are most 5 significant bits of a.

Store a in hl and multiply hl by `add hl, hl` twice to get 32Y.

`add hl, hl` is equivalent to $hl:=hl*2$. Proof of this can be seen in Homework 3.

For b, run `sr1` 3 times. `sr1` is right shift operation.

`sr1 a` is equivalent to $a:=a/2$. Proof of this can be seen in Homework 3.

Here, 32Y stored in hl, X stored in a, add them together. Note $a+l$ may cause overflow and borrow a bit in carry flag, use `adc a, h` to handle overflow case. Carry flag is reset during `and` instruction.

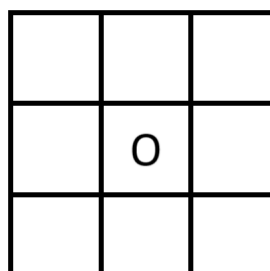
Don't forget to add `_SCRN0=$9800` at last.

.updatexxxbackground

Functionality: Given direction, update background map when moving xxx. These function are constructed for all cases of block and directions, $7*4=28$ in total.

Progress:

Consider the block the cursor pointed to on the screen



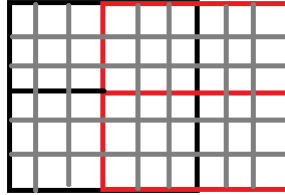
O denotes cursor. Assume position of cursor on screen is (x,y), then

$$\{(X, Y) | X \in \{x - 1, x, x + 1\}, Y \in \{y - 1, y, y + 1\}\}$$

is the area where a block take place of the screen.

If we want to move a small block (Bing) to another position, make the old position blank and new position be the moving block.

If it is a big block (Cao Cao), assume moving right, then its new position should be in the red part:



Where grey grid separate a big block into tiles.

We have hl pointed to the cursor by calling `GetTileByPixel` before. Use this pointer to access VRAM and increase or decrease hl to visit adjacent tiles. Note going right is increase 1, and going to the next line is increase 32, since line width is 32 in the screen.

There may be some new positions that coincide the old ones. Directly set them to new state as blank would be replaced afterwards.

Counter relative functions

`binToDec`

Functionality: Convert a binary value in the global variable "counter" to decimal number, store the reverse order in "buffer". That is, `[buffer]` stores units digit, `[buffer+1]` tens digit and `[buffer+2]` hundreds digit.

Progress:

Note that binary and decimal numbers can be converted from each other, while they're representing the exact same value.

By number decomposition, a number v can be represented as

$$v = 2^n b_n + 2^{n-1} b_{n-1} + \dots + 2b_1 + b_0 = 10^m d_m + 10^{m-1} d_{m-1} + \dots + 10d_1 + d_0$$

And $(b_n b_{n-1} \dots b_1 b_0)_2$ is binary representation, $(d_m d_{m-1} \dots d_1 d_0)$ is decimal representation of v .

We set m to 2 (and there's cases $d_2 = 0$ or $d_1 = 0$, where the value is ten value or unit value).

We want to obtain d_0 , d_1 and d_2 . By module 10, we have

$$v \equiv d_0 \pmod{10}$$

Hence

$$v = 10k + d_0$$

for some $k \in \mathbb{N}$ such that $d_0 < k$. Also d_0 is a digit, then $0 \leq d_0 \leq 9 < 10$.

To find d_0 , minus v for several time until current v is less than 10, at this state $v = d_0$.

Let register c be 0 at first. Every time we minus v by 10, we increase c by 1. When $v = d_0$, $c = k$.

Now

$$k = 10d_2 + d_1$$

Do the same "minus by 10" progress to k to get d_1 . Then d_2 . Store these digits in "buffer" variables.

copyDigitsRev

Functionality: Copy decimal digits from "buffer" to background map to update counter number displayed on the screen.

Progress:

Load digits in "buffer" to hl, and load counter position in the background map to de.

Note we're copying tiles to background map. Get the corresponding tile id by hl and replace hl.

Copy hl to [de] then increase hl, decrease de (since digits are stored in reverse order).

Loop 3 times to copy 3 digits.